

Primal-Dual Interior-Point Solver for Convex Quadratic Programming

Implementation with natID Sparse and Dense Matrices

Irfan Hadzic
Student index: 20013
`ihadzic1@etf.unsa.ba`
Numerical Optimization

September 2026

Abstract

This project implements a primal-dual interior-point solver for convex quadratic programming in C++20 using the natID linear algebra framework. The numerical core follows Mehrotra’s predictor-corrector method, assembles an augmented sparse Karush–Kuhn–Tucker system, and reuses its factorization for two direction computations per iteration. The application supports Matrix Market input, numerical diagnostics, automated tests, and a native graphical interface. The interface visualizes convergence and, for two-variable problems, objective contours, feasibility constraints, and the sequence of primal iterates. A dTwin reference adapter provides an additional solution comparison. The implementation targets educational convex problems with a regular KKT system; it is not a general-purpose solver with infeasibility certificates.

1 Introduction and objectives

Quadratic programming appears in estimation, control, and constrained optimization. It also provides a useful setting for studying the relationship between first-order optimality conditions, numerical linear algebra, and iterative algorithms. The aim of this project is to implement these components explicitly and make their behavior observable.

The proposal required natID dense and sparse matrix types, a Mehrotra predictor-corrector algorithm, sparse KKT assembly and factorization, Matrix Market input, and per-iteration residual diagnostics. These requirements form the numerical core. Interactive visualization, reference-solver comparison, and platform-specific installer workflows extend the original scope.

Nocedal and Wright provide the main mathematical reference [1]. As stated in my project proposal, I used the educational `cppipm` project as a conceptual reference [2]; the present implementation uses natID rather than Eigen for its production linear algebra.

2 Problem formulation

The solver addresses

$$\min_{x \in \mathbb{R}^n} f(x) = \frac{1}{2}x^T Qx + c^T x, \quad Ax = b, \quad Gx \leq h, \quad (1)$$

where $Q = Q^T \succeq 0$, $A \in \mathbb{R}^{p \times n}$, and $G \in \mathbb{R}^{m \times n}$. Convexity is an input assumption. The loader checks dimensions, finite values, and symmetry, but does not prove positive semidefiniteness by an eigenvalue decomposition.

Introducing slacks $s \geq 0$, equality multipliers y , and inequality multipliers $z \geq 0$ gives the KKT conditions

$$Qx + c + A^T y + G^T z = 0, \quad Ax - b = 0, \quad (2)$$

$$Gx + s - h = 0, \quad Sz = 0, \quad (3)$$

where $S = \text{diag}(s)$. The interior-point iteration keeps $s, z > 0$ and approaches complementarity as

$$\mu = \frac{s^T z}{m} \rightarrow 0 \quad (m > 0). \quad (4)$$

Primal iterates need not be feasible before convergence. A trajectory outside the shaded feasible region is therefore not, by itself, an implementation error.

3 Numerical method

3.1 Residuals and augmented KKT system

The residuals are

$$r_d = Qx + c + A^T y + G^T z, \quad (5)$$

$$r_e = Ax - b, \quad (6)$$

$$r_i = Gx + s - h. \quad (7)$$

For a chosen complementarity residual r_c , the Newton equations include $Z\Delta s + S\Delta z = -r_c$, with $Z = \text{diag}(z)$. Eliminating Δs and Δz yields

$$\begin{bmatrix} Q + G^T S^{-1} Z G & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = \begin{bmatrix} -r_d + G^T S^{-1} (r_c - Z r_i) \\ -r_e \end{bmatrix}. \quad (8)$$

The remaining directions follow from

$$\Delta s = -r_i - G\Delta x, \quad \Delta z = S^{-1}(-r_c - Z\Delta s). \quad (9)$$

Equation (8) describes the unregularized system; the implementation also provides a small configurable regularization for numerical robustness.

3.2 Mehrotra predictor-corrector iteration

The affine predictor uses $r_c = Sz$. Maximum positivity-preserving affine steps produce an estimate μ_{aff} , and the centering parameter is

$$\sigma = \text{clamp} \left[\left(\frac{\mu_{\text{aff}}}{\mu} \right)^3, 0, 1 \right]. \quad (10)$$

The corrected direction uses

$$r_c = Sz + \Delta S_{\text{aff}} \Delta z_{\text{aff}} - \sigma \mu \mathbf{1}. \quad (11)$$

Both solves use the same KKT matrix in an iteration. The code assembles and factorizes once, then replaces the right-hand side for the corrected solve.

For a positive vector v and direction Δv , the boundary step is

$$\alpha_{\max}(v, \Delta v) = \min_{j: \Delta v_j < 0} \frac{-v_j}{\Delta v_j}, \quad (12)$$

with no finite restriction when the index set is empty. Separate primal and dual steps are bounded by one and reduced using a fraction-to-boundary parameter, whose default is 0.995. This maintains positive slacks and inequality multipliers.

3.3 Stopping criteria and equality-only problems

The reported primal residual is $\max(\|r_e\|_\infty, \|r_i\|_\infty)$; the dual residual is $\|r_d\|_\infty$. Convergence requires all three scaled quantities

$$\frac{\max(\|r_e\|_\infty, \|r_i\|_\infty)}{1 + \max(\|b\|_\infty, \|h\|_\infty)}, \quad \frac{\|r_d\|_\infty}{1 + \|c\|_\infty}, \quad \frac{\mu}{1 + |f(x)|} \quad (13)$$

to be at most the selected tolerance. The raw duality gap $s^T z$ is also reported. Empty constraint groups contribute zero residual.

When $m = 0$, no complementarity iteration is needed. The solver instead solves the equality-constrained KKT equations directly. Termination statuses distinguish convergence, maximum iterations, invalid input, and numerical failure.

4 Software implementation

The project contains three principal targets:

- `natid_qp`: command-line solver and diagnostic output;
- `natid_qp_tests`: automated numerical and integration checks;
- `natid_qp_gui`: native interactive application.

Problem matrices and vectors use `dense::Matrix`; diagonal weighting uses `dense::DiagMatrix`.

The KKT system is assembled through `sparse::IMatrix::addTriple()` with `Symmetry::SymmetricIndef`.

Factorization uses `sparse::ISolver`, `SolverType::LDLT`, and `Pivoting::AlterMatrixIfIndefinite`.

The matrix and solver interfaces receive the same upper-triangular entries through a shared insertion helper. This allows inspection and nonzero statistics alongside numerical factorization. Although the final KKT system is sparse, input matrices and the augmented Hessian are held densely; the implementation should therefore not be interpreted as fully sparse end to end.

QP data can be loaded from a folder of Matrix Market files. CLI output includes iteration diagnostics, a text solution report, convergence CSV, and Matrix Market solution vectors. For small systems, diagnostic mode prints the KKT matrix before factorization and requests serialization after factorization. The precise representation of that latter output is determined by the `natID` API.

4.1 QP problem folders

The **QP problem** selector provides quick access to the built-in demos and valid problem folders in the packaged `data` directory. **Choose QP Folder** loads an external problem from a local directory. Each directory represents one QP and uses the following filenames:

File	Data	Dimensions	Requirement
Q.mtx	Quadratic objective matrix	$n \times n$	Required
c.mtx	Linear objective vector	$n \times 1$	Required
G.mtx	Inequality matrix	$m \times n$	Required
h.mtx	Inequality bounds	$m \times 1$	Required
A.mtx	Equality matrix	$p \times n$	Optional pair
b.mtx	Equality right-hand side	$p \times 1$	Optional pair

All files belong directly inside the selected folder. The equality files **A.mtx** and **b.mtx** must either both be present or both be absent. Vectors may also be stored as row vectors; the loader converts them to columns. Matrix Market coordinate and array storage are supported, and coordinate indices are one-based. Matrix dimensions must agree with the same values of n , m , and p . Selecting a problem loads and solves it, while external folders are added to the selector as custom entries.

5 Visualization and reference comparison

5.1 Synchronized animation

Each iteration record stores the primal vector $x^{(k)}$, objective value, residuals, complementarity measures, step lengths, and KKT statistics. The dashboard maintains one playback index shared by two tabs:

- **Objective Function:** for $n = 2$, objective contours, the iterate path, current point, constraint boundaries, and a converged solution marker; for higher dimensions, objective value versus iteration.
- **Residuals:** primal residual, dual residual, μ , and a tolerance reference line, with a selectable Y-axis mode.

The tolerance line is a visual reference: the actual stopping test uses the scaled quantities defined above, whereas the displayed residuals are raw values.

Previous/next step, play/pause, replay, and speed controls operate on the shared history. Changing tabs does not reset playback, and replay does not rerun the solver. Problem selection and tolerance changes are separate solver controls. Mouse-centered zoom and keyboard panning support inspection of closely spaced iterates. Ctrl+mouse wheel zooms around the cursor, while Ctrl+plus and Ctrl+minus provide keyboard zoom. The ordinary mouse wheel pans vertically, and Shift+mouse wheel pans horizontally. After clicking the canvas, arrow keys or W/A/S/D pan the plot; Ctrl+0 resets the view.

5.2 Tolerance and residual axis modes

The **Tolerance (threshold)** input sets the convergence threshold ε . The solver terminates successfully when all three scaled KKT measures defined in Section 3.3 are at most ε . A smaller threshold requests tighter convergence and can require more iterations; it does not directly bound the error in x . The GUI accepts values from 10^{-12} to 1 and initializes the threshold to 10^{-9} . **Run Again** recomputes the solution with the selected threshold, whereas **Play Again** replays the existing history.

The **Y-axis mode** selector affects only the residual view:

- **Linear:** plots the residual magnitude directly.

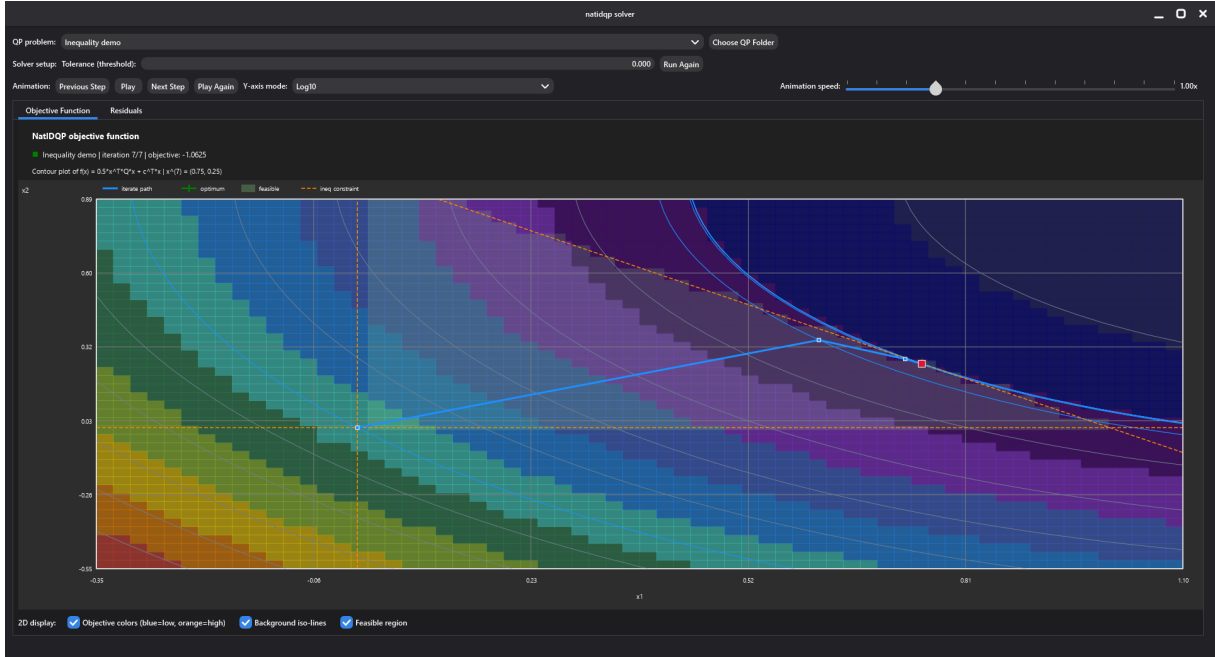


Figure 1: Two-variable inequality example: objective landscape and primal iterate path.

- **Log10**: plots its base-ten logarithm, making reductions across several orders of magnitude visible.
- **Accuracy (-log10)**: reverses the logarithmic sign, so increasing values indicate smaller residuals. A value of 6 corresponds to a residual magnitude of 10^{-6} , not necessarily six correct digits in the solution.
- **Enhanced accuracy**: applies the signed-square transformation $a|a|$ to $a = -\log_{10}(r)$ to emphasize differences near convergence.

For these logarithmic displays, residual magnitudes below 10^{-12} are clamped to 10^{-12} to keep zero values finite on the plot. The underlying solver values are unchanged. Axis transformations also apply to the tolerance reference line and do not modify the convergence test.

5.3 Visual accessibility and clutter reduction

Three independent checkboxes below the two-dimensional objective plot allow the display to be adapted to the amount of visual detail needed:

- **Objective colors** shows or hides the objective heatmap, with blue indicating lower values and orange indicating higher values.
- **Background iso-lines** shows or hides the faint gray contours while retaining the contours associated with solver iterates.
- **Feasible region** shows or hides the translucent feasible-set shading.

These options serve as visual accessibility aids by reducing overlapping layers and helping the reader focus on the trajectory, objective geometry, or constraints individually. Labels, point markers, and distinct constraint line patterns provide information in addition to color. Playback speed and manual stepping allow the animation to be followed at an individual pace. The heatmap uses a fixed objective scale during zooming, keeping its colors consistent with the solution and contour geometry.

5.4 dTwin comparison

The reference adapter creates an in-memory model of stationarity, feasibility, and smoothed Fischer–Burmeister complementarity equations through the native dTwin/modelSolver interface. It compares the resulting point and objective against the project solver, together with KKT residual information.

The reference solve first uses a neutral starting point. If necessary, it can retry using the project’s converged KKT point. The interface reports the start used: a successful warm-start check provides consistency evidence, but is not an independent convergence benchmark. No claim of comparative runtime superiority is made in this report.

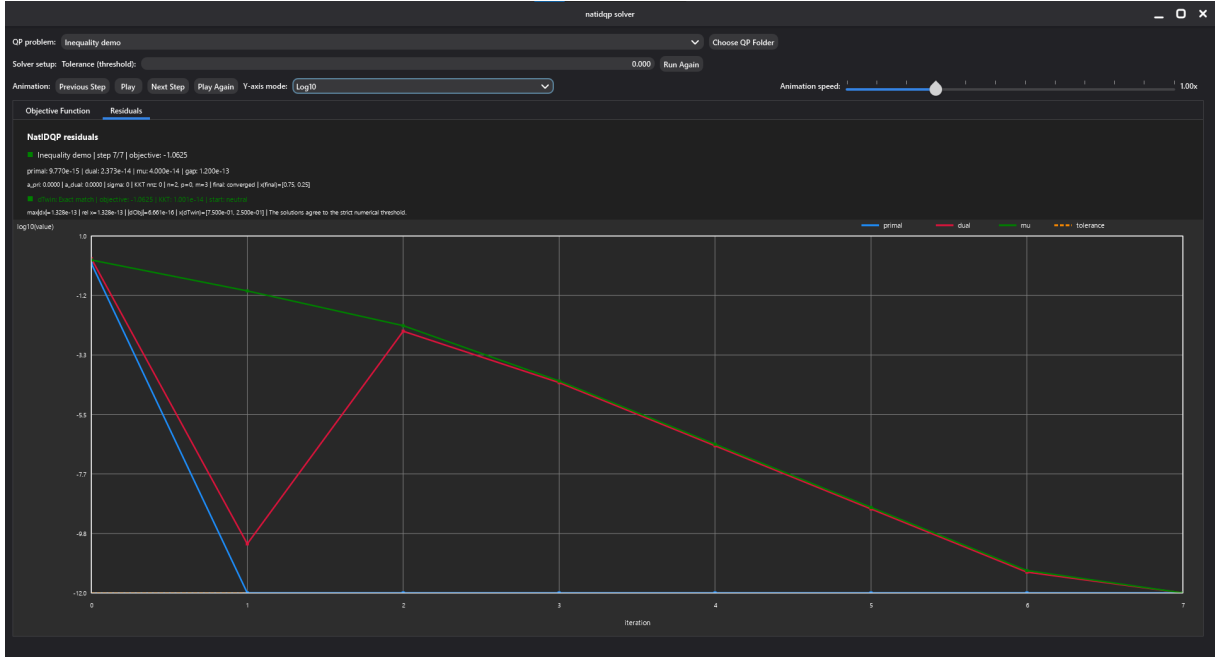


Figure 2: Residual convergence and reference-solver comparison for the inequality demo.

6 Verification and results

The project verification record documents C++20 builds against the production natID SDK on Windows, successful native tests, and a separate portable-backend test run. The portable backend is test-only and is not used to substantiate production performance claims. History checks verify that each stored primal vector has the expected size, reproduces its recorded objective, and ends at the returned solution.

Table 1 lists the known-solution cases recorded in the project. The values are expected optima used for numerical validation, rather than a newly collected benchmark dataset for this report.

Case	Expected x^*	Expected $f(x^*)$
Inequality demo	(0.75, 0.25)	-1.0625
Equality with nonnegativity	(0.25, 0.75)	-0.125
Equality-only KKT case	(1, 0)	—

Table 1: Known-solution test cases.

The verification record also includes Matrix Market versions of the examples, rejection of a nonsymmetric Q , reference-comparison classification tests, and sanitizer checks. An earlier generated case with $n = 20$ and $m = 21$ converged in seven iterations at the default tolerance, with the final dual residual below 9×10^{-10} . This is a recorded functional check, not evidence of large-scale performance.

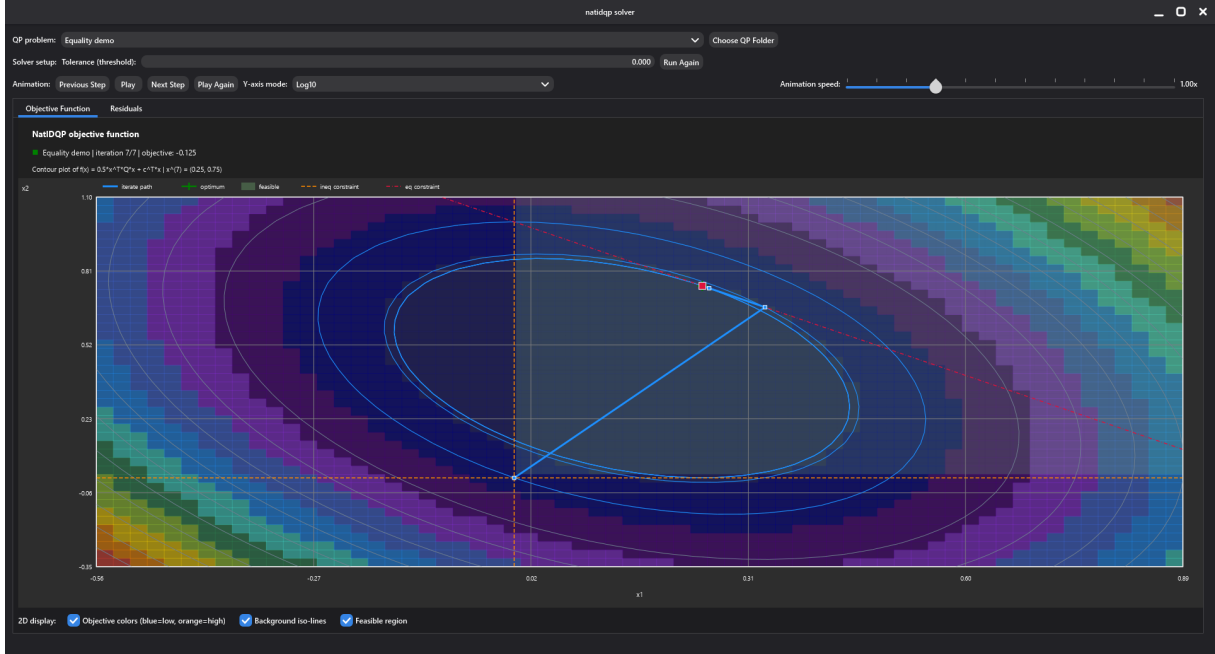


Figure 3: Equality-constrained example and convergence toward the feasible optimum.

These checks support correctness on the tested problems. They do not establish robustness for all convex QPs, particularly degenerate, badly scaled, infeasible, or unbounded cases. No new timing study was performed when preparing this report.

7 Packaging and reproducibility

I automated building and packaging with GitHub Actions for Windows, macOS, and Linux. All three platform builds completed successfully, and the installers are available through the project’s GitHub Releases. The packages include the application resources and example QP data, allowing the supplied problems to be explored directly after installation.

For development, the project can also be built from source using C++20, CMake, and a compatible natID SDK with the corresponding platform libraries. The repository documentation describes the configuration and build commands. The CLI exports convergence histories and solution vectors, allowing runs with the same problem data and tolerance to be compared numerically.

8 Limitations and future work

The present scope assumes convex input and a sufficiently regular KKT system. It does not provide infeasibility or unboundedness certificates. Dense storage of input matrices limits scalability even though the KKT solve uses a sparse interface. Additional work could include sparse

```

Problem dimensions: n=2, p=0, m=3

iter      objective      primal_res      dual_res      mu      a_pri      a_dual      nnz
-----
KKT matrix before factorization:
Output_TEST:KKT=sparse(2,2);
KKT(1,1)=2.500000000100000e+00;
KKT(1,2)=5.000000000000000e-01;
KKT(2,2)=2.500000000100000e+00;
KKT matrix after natID LDLT factorization:
KKT_factorized=sparse(2,2);
KKT_factorized(1,1)=2.500000000100000e+00;
KKT_factorized(1,2)=5.000000000000000e-01;
KKT_factorized(2,2)=2.500000000100000e+00;

      0      0.00000e+00      1.00000e+00      1.50000e+00      1.33333e+00      1.0000      1.0000      3
      1     -1.01562e+00      0.00000e+00      6.13241e-11      1.00107e-01      1.0000      0.9703      3
      2     -1.05915e+00      0.00000e+00      3.40017e-03      5.41919e-03      0.9983      0.9967      3
      3     -1.06249e+00      0.00000e+00      4.57336e-05      5.23008e-05      0.9950      0.9950      3
      4     -1.06250e+00      1.11022e-16      2.28848e-07      2.61382e-07      0.9950      0.9950      3
      5     -1.06250e+00      0.00000e+00      1.14424e-09      1.30690e-09      0.0000      0.0000      0

Solver status: converged
Message: All scaled KKT residuals satisfy the requested tolerance.
Iterations: 5
Objective: -1.062499999633e+00
x = [7.499999938943e-01; 2.500000056162e-01]
s (slack) = [4.895330847870e-10; 7.499999938943e-01; 2.500000056162e-01]
z (inequality dual) = [7.500000073640e-01; 1.140772774065e-10; 1.387199656110e-08]
Final primal residual (inf): 0.000000000000e+00
Final dual residual (inf): 1.144242837365e-09
Final duality gap: 3.920706992737e-09
Final complementarity mu: 1.306902330912e-09

```

Figure 4: Command-line iteration diagnostics and final primal-dual solution.

input retention, equilibration, iterative refinement, presolve, symbolic factorization reuse across iterations, and a homogeneous self-dual formulation.

The two-dimensional objective display is an educational visualization. For higher-dimensional problems, an objective-versus-iteration plot avoids presenting a misleading two-dimensional projection as the full objective. Further experiments could compare accuracy and runtime across larger families of problems, with clearly controlled initialization and stopping criteria.

9 Conclusion

The project implements the proposed primal-dual solver with natID matrix types, sparse augmented KKT assembly, LDLT factorization, a two-phase Mehrotra direction calculation, positivity-preserving steps, and residual-based termination. Matrix Market input and numerical output make the solver usable outside the GUI. Automated checks and known-solution examples provide evidence for the implemented numerical paths, while synchronized visualization and the dTwin adapter extend the original educational objectives.

References

- [1] J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006, Chapter 16.
- [2] Y. Yan, *cppipm*, educational interior-point implementation. <https://github.com/YimingYAN/cppipm>.

- [3] natID framework, SDK headers and examples used by the project. <https://github.com/idzafic/natID>.